

Image Evaluation and Quality Scoring Framework

1. Assignment Objective

The objective of this assignment is to design and implement a **comprehensive, offline evaluation and quality scoring framework** for a **Retrieval-Augmented Generation (RAG)** system operating in a **closed-domain environment**. The system must use a **single PDF document as its exclusive knowledge source** and must not rely on any external APIs or online large language models.

The primary focus of this task is not only to retrieve and generate answers, but to **systematically evaluate the reliability, accuracy, grounding, efficiency, and robustness** of the RAG system. The final outcome should be a **single composite RAG Quality Score (0–100)** that is derived from clearly defined and explainable evaluation metrics.

2. Constraints

- The entire system must operate **fully offline**, without the use of cloud services or hosted language model APIs.
 - Only **one PDF document(shall be provided)** may be used as the knowledge source, and all answers must be derived exclusively from its content.
 - All processing, evaluation, and scoring must be **deterministic and reproducible**.
 - The system must be explicitly designed and evaluated as a **closed-domain RAG system**.
 - **Finetuning** the model is an optional step that can be performed to enhance the system's response.
-

3. Input Data

3.1 Source Document

The candidate must select a single PDF document (given). This document will serve as the **sole authoritative source of information** for the entire system.

All data used by the RAG pipeline—including retrieval, answer generation, and evaluation—must originate strictly from this document.

4. System Requirements

4.1 Document Processing

The system must extract the **complete textual content** of the PDF, including all relevant sections required to answer evaluation queries. The extracted content should be structured in a way that preserves essential contextual information such as page numbers or section identifiers where possible.

The processed document must be segmented into **retrievable units** that are suitable for both retrieval and downstream evaluation. The segmentation strategy should be clearly justified in the final report.

4.2 Evaluation Dataset Construction

The candidate must construct an evaluation dataset derived entirely from the source PDF. This dataset must consist of **40–60 carefully designed natural language questions** covering a broad range of difficulties and reasoning requirements.

The dataset should include:

- Fact-based questions requiring precise retrieval
- Analytical questions requiring synthesis across multiple sections
- Multi-step reasoning questions within the document
- Questions for which the correct response is that the information is **not present** in the document

For each question, the candidate must provide either a **reference answer** or a **clearly defined evaluation rubric**. For a subset of the questions, the candidate must also identify which parts of the document are considered relevant for answering the question.

4.3 Retrieval Component

The retrieval component must be designed to accept natural language questions and retrieve the **most relevant document segments** from the processed PDF. The system must retrieve a configurable number of top-ranked segments and maintain detailed logs of retrieval results to support quantitative evaluation and error analysis.

4.4 Answer Generation

Answer generation must be strictly constrained to the retrieved document segments. The system must avoid introducing unsupported facts or external knowledge. When the document does not contain sufficient information to answer a question, the system must explicitly acknowledge this limitation rather than attempting to fabricate an answer.

The approach used for answer generation must be clearly described and justified.

5. Evaluation and Metrics: Programmatic Calculation

The system must calculate the following six metrics using raw mathematical implementations (NumPy is permitted; higher-level ML libraries like `sklearn` are prohibited for the scoring logic).

5.1 Contextual Precision

Definition: Evaluates if the most relevant chunks are ranked higher in the retrieved list.

Programmatic Calculation:

$$\text{Contextual Precision} = \frac{\sum_{k=1}^n (P@k \times \text{rel}_k)}{\text{Total Relevant Segments}}$$

- **Implementation:** Iteratively calculate Precision@k for each rank \$k\$ where the segment is relevant.
- **Requirement:** Use binary relevance (1 if segment contains part of the answer, 0 otherwise).

5.2 Contextual Recall

Definition: Measures if the retrieved context contains all the necessary information to produce the ground truth answer.

Programmatic Calculation:

$$\text{Contextual Recall} = \frac{|\text{Attributes in Ground Truth found in Context}|}{|\text{Total Attributes in Ground Truth}|}$$

- **Implementation:** Use a local LLM to decompose the ground truth into N atomic statements and check their presence in the retrieved context via string matching or local similarity.

5.3 Contextual Relevancy

Definition: The signal-to-noise ratio of the retrieved context.

Programmatic Calculation:

$$\text{Contextual Relevancy} = \frac{\text{Number of relevant sentences in Context}}{\text{Total sentences in Context}}$$

- **Implementation:** Parse retrieved segments into sentences. Use a local embedding model to compute **Cosine Similarity** (see formula below) between each sentence and the query. If $\text{Sim} > \text{threshold}$, the sentence is relevant.

5.4 Faithfulness (Groundedness)

Definition: Ensures the generated answer is derived strictly from the retrieved context.

Programmatic Calculation:

$$\text{Faithfulness} = \frac{\text{Number of claims in Answer supported by Context}}{\text{Total number of claims in Answer}}$$

- **Implementation:** Use a local LLM to list all claims in the generated response. For each claim, perform a check against the context.

5.5 Answer Relevancy

Definition: How well the generated answer addresses the user's query (ignoring the ground truth).

Programmatic Calculation: Use **Cosine Similarity** between the query embedding vector ($\mathbf{q}$) and the answer embedding vector ($\mathbf{a}$):

$$\text{Score} = \frac{\mathbf{q} \cdot \mathbf{a}}{\|\mathbf{q}\| \|\mathbf{a}\|} = \frac{\sum_{i=1}^n q_i a_i}{\sqrt{\sum_{i=1}^n q_i^2} \sqrt{\sum_{i=1}^n a_i^2}}$$

- **Requirement:** Implement this using `numpy.dot` and `numpy.linalg.norm`.

5.6 Answer Correctness (Semantic + Factual)

Definition: Comparison between the generated answer and the Reference Answer.

Programmatic Calculation: A weighted combination of:

1. **Semantic Similarity:** Cosine similarity of the two answer embeddings.
2. **Factual Overlap:** Calculate the **ROUGE-L** (Longest Common Subsequence) score manually:

$$\text{ROUGE-L} = \frac{\text{LCS}(\text{Gen}, \text{Ref})}{\text{Length}(\text{Ref})}$$

6. Composite Quality Scoring (0–100)

The final **RAG Quality Score** must be an aggregate of the above six metrics. The candidate must justify the weights assigned to each.

Scoring Methodology Template

The candidate must implement a function `calculate_composite_score()` that applies the following weights:

Component	Metric	Weight
Retrieval (40%)	Contextual Precision	15%
	Contextual Recall	15%
	Contextual Relevancy	10%
Generation (60%)	Faithfulness	25%
	Answer Relevancy	20%
	Answer Correctness	15%

Penalty Logic:

- If **Faithfulness < 0.5**, a "Hallucination Penalty" of **-20 points** is applied to the final score.
- If **Contextual Recall is 0**, the generation score is capped at **30** (as the answer is likely fabricated).

Implementation Note for the Candidate:

All similarity calculations must be performed using **NumPy**. You are required to log the raw vectors and the dot-product results for at least three sample queries to prove the programmatic accuracy of your custom scoring functions.

7. Analysis and Reporting

The final submission must include a detailed technical report that describes:

- the overall system architecture and design decisions
 - the structure and composition of the evaluation dataset
 - the chosen evaluation metrics and scoring methodology
 - quantitative results with metric-level breakdowns
 - qualitative analysis of failure cases
 - limitations of the current approach and potential improvements
-

8. Deliverables

1. Well-structured and documented source code
 2. Evaluation dataset including questions and references
 3. Evaluation outputs and scoring results
 4. Final technical report in PDF or Markdown format
-

9. Evaluation Criteria

Submissions will be evaluated based on:

- depth and rigor of the evaluation framework

- clarity and soundness of metric definitions
- quality of technical reasoning and trade-off analysis
- robustness of the system design
- handling of grounding and hallucination
- clarity, structure, and professionalism of documentation

Evaluation Metrics are RAGAs:

To fulfill your assignment's requirement for a comprehensive, offline evaluation framework, here are the **six key RAG metrics** derived from the RAGAS framework.

1. Faithfulness (Generation)

- **Definition:** Measures if the generated answer is derived *only* from the retrieved context (no hallucinations).
- **The Logic:** You must verify if every claim in the answer is present in the context.
- **Programmatic Calculation:**

$$\text{Faithfulness Score} = (\text{Number of claims supported by Context}) / (\text{Total number of claims in Answer})$$
- **Implementation:**
 1. Split the generated answer into individual sentences (claims).
 2. Check if each sentence is semantically present in the retrieved chunks.
 3. If a claim is not found in the PDF context, it is a hallucination.

2. Answer Relevancy (Generation)

- **Definition:** Evaluates how well the answer addresses the user's question, regardless of whether it's "correct."
- **The Logic:** A highly relevant answer directly targets the prompt.
- **Programmatic Calculation (Cosine Similarity):**
$$\text{Score} = (\text{Query_Vector} \cdot \text{Answer_Vector}) / (||\text{Query_Vector}|| \times ||\text{Answer_Vector}||)$$
- **Implementation:**
 1. Get embeddings for the Query (\$Q\$) and Answer (\$A\$).
 2. Use NumPy: $\text{np.dot}(Q, A) / (\text{np.linalg.norm}(Q) * \text{np.linalg.norm}(A))$.

3. Contextual Precision (Retrieval)

- **Definition:** Measures the **ranking quality** of your retrieval. Are the most relevant chunks at the very top?
- **The Logic:** If the "answer chunk" is at index 5 but index 1–4 are useless, your precision is low.
- **Programmatic Calculation:**
$$\text{Contextual Precision} = [\text{Sum of } (\text{Precision@k} \times \text{relevance_k})] / \text{Total Relevant Segments}$$
- **Implementation:**
 1. Iterate through the top- k retrieved chunks.
 2. At each rank k , if the chunk is relevant (contains ground truth keywords), calculate Precision@k (Relevant found so far / k).
 3. Average these precision values for all relevant chunks.

4. Contextual Recall (Retrieval)

- **Definition:** Measures the **completeness** of the context. Did the system find *all* the facts needed to answer?
- **The Logic:** If the Ground Truth has 3 facts but your chunks only have 2, recall is $2/3$.
- **Programmatic Calculation:**
$$\text{Contextual Recall} = (\text{Number of Ground Truth facts found in Context}) / (\text{Total facts in Ground Truth})$$
- **Implementation:**
 1. Break the "Reference Answer" into atomic facts.
 2. For each fact, loop through the retrieved chunks to check for a match.

5. Contextual Relevancy (Retrieval)

- **Definition:** Measures the "signal-to-noise" ratio. Is the retrieved text mostly useful, or full of filler?
- **The Logic:** A 1000-word chunk where only 1 sentence is useful has low relevancy.
- **Programmatic Calculation:**
$$\text{Contextual Relevancy} = (\text{Number of relevant sentences in Context}) / (\text{Total sentences in Context})$$
- **Implementation:**
 1. Parse retrieved chunks into individual sentences.
 2. Use NumPy cosine similarity (see 5.2) to compare each sentence against the query.
 3. Count sentences where $\text{Similarity} > 0.4$ as relevant.

6. Answer Correctness (End-to-End)

- **Definition:** Compares the generated answer directly against the Ground Truth (Reference Answer).
- **The Logic:** It combines factual overlap with semantic meaning.
- **Programmatic Calculation:**

$$\text{Correctness Score} = (\text{Weight1} \times \text{Semantic Similarity}) + (\text{Weight2} \times \text{Factual Similarity})$$
- **Implementation:**
 1. **Semantic Similarity:** Cosine similarity (NumPy) between the generated and reference answers.
 2. **Factual Similarity:** Count overlapping unique keywords (unigrams) or n-grams.
 3. A common weight is **0.5** for each.

The Composite RAG Quality Score (0–100)

To get your final score, you must aggregate these metrics using a weighted function.

Metric	Weight	Category
Faithfulness	25%	Generation
Contextual Recall	20%	Retrieval
Answer Relevancy	15%	Generation
Contextual Precision	15%	Retrieval
Answer Correctness	15%	End-to-End
Contextual Relevancy	10%	Retrieval

Final Calculation:

$$\text{Total Score} = (\text{Metric}_1 \times 0.25) + (\text{Metric}_2 \times 0.20) \dots \times 100$$

10. Expected Outcome

A robust and well-reasoned **offline RAG evaluation and scoring framework** that demonstrates the candidate's ability to design, analyze, and evaluate complex NLP systems under realistic constraints, with a strong emphasis on reliability, explainability, and responsible AI practices.